TURISMO UQ

Valentina Gonzalez Diaz

Valentina Rodriguez Castro

Raquel Lopez Aristizabal

Código:

1094898396

1016833647

1091884575



Profesor: Carolina Londoño Idarraga

Universidad del Quindío

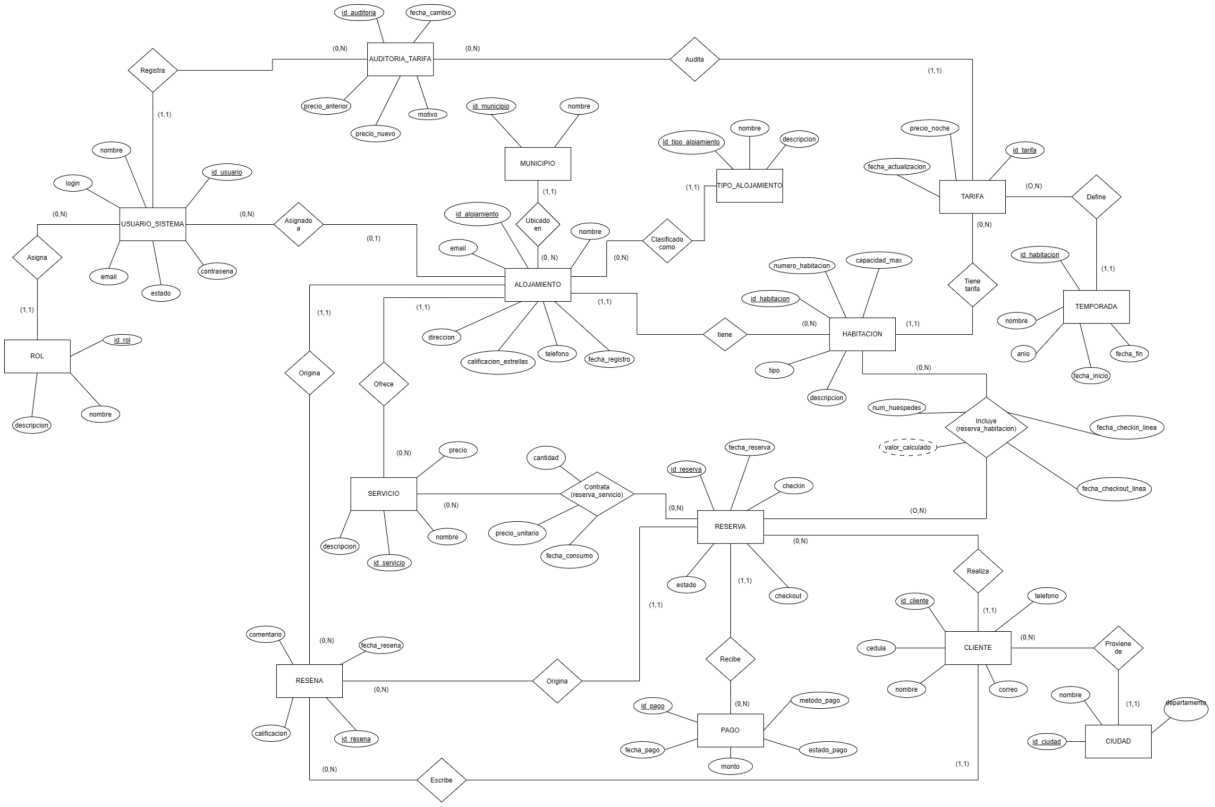
Facultad de Ingeniería

Programa Ingeniería de Sistemas y Computación

Bases de Datos II

Armenia, 2026

DIAGRAMA MER:



Para mejor visualización presione [Diagrama MER]

DECISIONES DE DISEÑO:

Decisión 1: Una reserva puede incluir varias habitaciones

Modelo: entidad asociativa RESERVA_HABITACION entre RESERVA y HABITACION

- RESERVA guarda los datos generales: id_reserva, id_cliente (FK), fecha_reserva, estado (pendiente, confirmada, cancelada, completada) y el rango global checkin / checkout, útil para listar y filtrar rápido. El rango global cubre todas las líneas: checkin es el mínimo de fecha_checkin_linea y checkout es el máximo de fecha_checkout_linea.

- RESERVA_HABITACION guarda una fila por cada habitación incluida, con sus propias fechas de check-in/check-out.

Atributos: id_reserva_hab, id_reserva (FK), id_habitacion (FK), fecha_checkin_linea, fecha_checkout_linea, num_huespedes, valor_calculado..

Justificación del diseño: Si las fechas se manejan únicamente en la tabla principal RESERVA, sería imposible modelar escenarios donde miembros de un mismo grupo o familia llegan o se van en días diferentes dentro del mismo trámite de reserva. Ubicar las fechas en RESERVA_HABITACION otorga independencia a cada línea para efectos de disponibilidad y tarifas, manteniendo la cohesión del grupo bajo una misma transacción de pago.

Restricciones que se van a implementar:

- UNIQUE para id_reserva, id_habitacion. Una habitación aparece una sola vez por reserva.
- CHECK (fecha_checkout_linea > fecha_checkin_linea) y CHECK (num_huespedes > 0).
- Como HABITACION tiene capacidad_max, num_huespedes no debe superarla por lo que hay que validarlo con un trigger.

Regla de no-solape: Una habitación no puede tener dos líneas con rangos de fechas solapados si ambas pertenecen a reservas activas, es decir, en estado pendiente, confirmada o completada. Una reserva cancelada no ocupa la habitación. Como CHECK o UNIQUE no pueden validarla, se implementa con el procedimiento sp_crear_reserva y con un trigger sobre RESERVA_HABITACION.

En la entrega 2 los disparadores de fila validan, al registrar una línea de reserva: que la habitación no esté ocupada por otra reserva activa (trg_no_solape_reserva), que todas las líneas de una reserva sean del mismo alojamiento (trg_reserva_un_alojamiento), que los huéspedes no superen la capacidad y que las fechas de la línea queden dentro del rango de la reserva.

Decisión 2 : Una estadía puede atravesar dos temporadas

Modelo: no se guarda un "precio de la estadía" fijo en ningún lado. Se guarda:

- TEMPORADA: id_temporada, nombre (alta/media/baja), anio, fecha_inicio, fecha_fin (donde fecha_fin > fecha_inicio).

- TARIFA: id_tarifa, id_habitacion (FK), id_temporada (FK), precio_noche (mayor a 0), fecha_actualizacion. Hay una fila por cada combinación de habitación y temporada (UNIQUE (id_habitacion, id_temporada).

Lógica de cálculo: El valor de la estadía se calcula dinámicamente noche por noche del rango [fecha_checkin_linea, fecha_checkout_linea -1] de la línea se identifica la temporada con un BETWEEN, se toma el precio_noche de TARIFA para esa habitación y temporada, y se suman las noches. El día del checkout no se cobra.

Justificación del diseño: Almacenar un precio único por reserva provocaría inconsistencias y pérdida de precisión en estadías que cruzan fechas críticas (por ejemplo, del 20 al 26 de diciembre, donde el 24 cambia de temporada media a alta). Al desglosar el cálculo diario cruzando TEMPORADA y TARIFA, permite variaciones estacionales sin requerir modificaciones en la estructura relacional.
Las temporadas no deben solaparse y deben cubrir todas las fechas reservables ya que si no se hace una noche podría tener dos precios o ninguno. En la Entrega 2 se valida con el disparador trg_temporada_sin_solape (ORA-20017), y al calcular cada estadía con fn_valor_estadia (ORA-20006 y ORA-20007).

Conexión entre decisiones:

- El atributo valor_calculado en RESERVA_HABITACION almacena el resultado de calcular las tarifas diarias sobre las fechas de esa línea específica. Si después se cambia una tarifa, las reservas ya calculadas no se alteran y los cambios de precio quedan registrados en AUDITORIA_TARIFA.

- RESERVA_SERVICIO guarda precio_unitario de forma similar, como copia del precio del servicio al momento del consumo.
- El valor total de la reserva es la suma del valor_calculado (todas las lineas) más la sumatoria de cantidad por precio_unitario de RESERVA_SERVICIO. Los pagos se comparan contra este total.

REGLAS DE NEGOCIO:

1. **Reembolso por cancelación:** Si el cliente cancela con más de 5 días de anticipación al check-in, se le reembolsa el 80 % de lo pagado. Con 5 días o menos no hay reembolso. La reserva pasa a estado ‘cancelada’ y se registra un pago ‘reembolsado’ por el 80 % del monto pagado con éxito.

2. **Reseñas solo de estadías completadas:** Un cliente solo puede reseñar un alojamiento sí tiene al menos una reserva ‘completada’ allí. La reseña debe apuntar a esa reserva. (trg_resena_completada, ORA-20015).

3. **Reserva pagada:** Una reserva está pagada solo si la suma de sus pagos ‘exitoso’ es mayor o igual al total de la sumatoria del valor calculado más la sumatoria de cantidad por precio unitario. Los pagos fallidos, pendientes o reembolsados no suman.

4. **Una reserva pertenece a un solo alojamiento:** Todas las habitaciones de una misma reserva deben ser del mismo alojamiento. (trg_reserva_un_alojamiento, ORA-20011)

5. **Servicios del alojamiento donde se hospeda:** Solo se pueden agregar a una reserva servicios que pertenezcan al alojamiento de sus habitaciones. (trg_servicio_del_alojamiento, ORA-20012)

6. **No solapar la ocupación:** Una habitación no puede tener líneas con fechas solapadas en reservas activas (pendiente, confirmada o completada); solo las canceladas liberan la habitación. Las noches se cuentan en el rango [checkin, checkout): desde la noche del check-in hasta la noche anterior al check-out, así que el día de salida queda libre para el siguiente ingreso. Se incluye la pendiente para que dos clientes no puedan reservar la misma habitación mientras una reserva espera confirmación.

7. **Capacidad máxima y fechas de línea:** El número de huéspedes de una línea no puede superar la capacidad máxima de la habitación. Las fechas de cada línea deben quedar dentro del rango general (checkin, checkout) de la reserva. (trg_linea_capacidad_fechas, ORA-20013 y ORA-20014)

8. **Temporadas sin solapamiento y sin huecos:** Para cada noche reservable existe exactamente una temporada y una tarifa para la habitación. Sin tarifa vigente no se puede reservar. Si falta temporada o tarifa para alguna noche, fn_valor_estadia lanza ORA-20006; si dos temporadas cubren la misma noche, lanza ORA-20007. Además, el disparador trg_temporada_sin_solape impide registrar una temporada que se cruce con otra (ORA-20017).

9. **Los usuarios internos tienen un rol:** El administrador de plataforma gestiona todos los alojamientos y no está atado a ninguno. El encargado pertenece a un alojamiento específico y solo puede gestionar lo de este alojamiento. Un usuario inactivo no puede operar.

10. **Precios históricos:** Los precios que ya se cobraron no cambian cuando cambian las tarifas o los precios de los servicios. Al reservar el valor de la línea se guarda el valor calculado, y al contratar un servicio se guarda el precio de ese momento. Los cambios en tarifa quedan registrados en auditoría tarifa, con quién los hizo, cuándo, el precio anterior y el nuevo.

CAPA PL/SQL – ENTREGA 2:

Orden de ejecución de los scripts

Todos los scripts están en la carpeta Scripts del repositorio y se ejecutan de arriba abajo, en este orden:

1. **ScriptDDL-TurismoUQ.sql:** tablas, llaves y restricciones.
2. **Carga_TurismoUQ.sql:** datos con el volumen mínimo.
3. **Consultas_Analisis_TurismoUQ.sql:** las siete consultas de la Entrega 1.
4. **E2_01_PaqueteTurismouq.sql:** paquete pkg_turismouq con la función, los procedimientos y el cursor.
5. **E2_02_Disparadores.sql:** disparadores de auditoría, de no solape y de reglas de negocio.
6. **E2_03_Tablespace_VistaMaterializada.sql:** tablespace del histórico y vista materializada. Su parte A se ejecuta una sola vez en la conexión SYSTEM.
7. **E2_04_Pruebas.sql:** evidencia de pruebas de todo lo anterior.

1. Paquete pkg_turismouq

El paquete agrupa la función y los dos procedimientos de la entrega. También declara las excepciones propias con PRAGMA EXCEPTION_INIT, para que quien lo llame pueda capturarlas por nombre (por ejemplo, pkg_turismouq.e_hab_no_disponible).

fn_valor_estadia(id_habitacion, fecha_in, fecha_out): calcula el valor de una estadía noche por noche. Con CONNECT BY genera una fila por cada noche, desde la fecha de entrada hasta la noche anterior a la salida (el día del check-out no se cobra). Cada noche se une con la TEMPORADA que la contiene (BETWEEN fecha_inicio AND fecha_fin) y con la TARIFA de esa habitación en esa temporada, y se suman los precios. Así, una estadía del 14 al 18 de diciembre de 2026 cobra dos noches con la tarifa de temporada baja (14 y 15) y dos con la de temporada alta (16 y 17).

La función también revisa los datos que usa: si una noche queda cubierta por dos temporadas lanza ORA-20007, y si alguna noche no tiene temporada o tarifa lanza ORA-20006. Además rechaza fechas inválidas (ORA-20001) y habitaciones que no existen (ORA-20004).

sp_crear_reserva: registra una reserva de una habitación. Valida en este orden: que el check-out sea posterior al check-in (ORA-20001), que el cliente exista (ORA-20005), que la habitación exista (ORA-20004), que haya al menos un huésped (ORA-20008) sin superar la capacidad máxima (ORA-20003), y que la habitación no esté ocupada en esas fechas por otra reserva activa (ORA-20002). Si todo está bien, calcula el valor con fn_valor_estadia e inserta el encabezado en RESERVA, en estado pendiente, y la línea en RESERVA_HABITACION.

Las dos inserciones quedan protegidas por un SAVEPOINT: si la segunda falla, se deshace también la primera y no queda una reserva sin habitación. El procedimiento no hace COMMIT; lo decide quien lo llama.

sp_liquidacion_mensual(anio, mes) y el cursor explícito: es el proceso masivo de liquidación mensual por alojamiento. El cursor explícito c_liq recibe como parámetros el primer día del mes y el primer día del mes siguiente, y devuelve cada alojamiento con dos totales: el valor de las estadías confirmadas o completadas que empiezan en el mes y el valor de los servicios consumidos en el mes.

El procedimiento recorre el cursor con OPEN, FETCH, EXIT WHEN %NOTFOUND y CLOSE, y guarda en la tabla LIQUIDACION_MENSUAL los alojamientos que tuvieron movimiento. Antes de empezar borra la liquidación anterior de ese mes, así que el proceso se puede repetir sin duplicar filas. Si el mes o el año no son válidos, lanza ORA-20020.

2. Disparadores

trg_auditoria_tarifa (nivel de sentencia): registra en AUDITORIA_TARIFA quién cambió una tarifa, cuándo, el precio anterior y el nuevo. La dificultad es que un disparador de sentencia se ejecuta una sola vez por instrucción y no tiene acceso a :OLD ni a :NEW, así que por sí solo no conoce el precio anterior.

Para resolverlo se creó la tabla de apoyo TARIFA_PRECIO_PREVIO, que guarda el último precio conocido de cada tarifa. Al terminar cada INSERT, UPDATE o DELETE sobre TARIFA, el disparador compara los precios actuales con esa copia. Donde son distintos registra el cambio (precio anterior = copia, precio nuevo = TARIFA) y luego actualiza la copia. Así, un solo UPDATE que cambie tres tarifas deja tres filas de auditoría.

Para saber quién hizo el cambio, se busca en USUARIO_SISTEMA un login igual al usuario de Oracle conectado. Si no existe, el cambio se asigna al administrador de la plataforma (admin.turismo), y el usuario real de Oracle queda escrito en la columna motivo. Si tampoco existe el administrador, lanza ORA-20016.

trg_no_solape_reserva (nivel de fila): antes de insertar cada línea en RESERVA_HABITACION, revisa que la habitación no tenga otra línea con fechas cruzadas en una reserva activa (pendiente, confirmada o completada). Dos rangos se cruzan si la entrada nueva es anterior a la salida existente y la salida nueva es posterior a la entrada existente. Si hay cruce, lanza ORA-20010 y la línea no se guarda. Las reservas canceladas no bloquean la habitación. Se probó con un caso real: una línea que ya existe en los datos cargados.

Otros disparadores de reglas de negocio:

- **trg_reserva_un_alojamiento:** todas las habitaciones de una reserva son del mismo alojamiento (ORA-20011).
- **trg_servicio_del_alojamiento:** los servicios de una reserva son del alojamiento donde se hospeda (ORA-20012).
- **trg_linea_capacidad_fechas:** los huéspedes no superan la capacidad (ORA-20013) y las fechas de la línea están dentro de las de la reserva (ORA-20014).
- **trg_resena_completada:** solo se reseña una estadía completada del mismo cliente en ese alojamiento (ORA-20015).
- **trg_lineas_validar_upd y trg_no_solape_reactivar:** repiten las validaciones de no solape y de un solo alojamiento cuando se modifica una línea o se reactiva una reserva cancelada (ORA-20010 y ORA-20011). trg_lineas_validar_upd es un disparador, para evitar el error de tabla mutante (ORA-04091).
- **trg_temporada_sin_solape:** impide registrar una temporada que se cruce con otra (ORA-20017).

3. Tablespace TS_HISTORICO para el histórico de reservas

Se creó el tablespace TS_HISTORICO con su propio datafile (ts_historico01.dbf: 20 MB iniciales que crecen de 5 en 5 MB hasta 100 MB). En él vive la tabla RESERVA_HISTORICO, particionada por rango según el año del check-in: la partición p_2024 guarda las reservas de 2024 y p_2025 las de 2025. Cuando una consulta filtra por fecha, Oracle solo lee la partición de ese año (poda de particiones), lo que se comprueba con EXPLAIN PLAN.

Justificación: las reservas del año en curso cambian todo el tiempo (se crean, se confirman, se cancelan y reciben pagos), mientras que las de años cerrados ya no cambian y solo se consultan para los reportes de gerencia. Separarlas en su propio tablespace permite:

- **Controlar el espacio:** el histórico crece con su propio límite y no le quita espacio a USERS, donde están las tablas operativas.
- **Respaldarlo aparte:** el histórico se respalda una vez, después del cierre del año, y no en cada respaldo diario.
- **Protegerlo:** después del cierre se puede poner en solo lectura (ALTER TABLESPACE ts_historico READ ONLY) para que nadie lo modifique por error.
- **Separar la carga del disco:** en un servidor real el datafile puede ir en otro disco, sin competir con las transacciones del día.

La tabla es una copia: las reservas no se borran de RESERVA, porque las consultas de la Entrega 1 y la vista materializada usan los tres años, y PAGO, RESENA, RESERVA_HABITACION y RESERVA_SERVICIO dependen de ellas por llave foránea. Al cerrar cada año se agrega una partición nueva y se copian las reservas de ese año.

4. Vista materializada MV_OCUPACION_MENSUAL

Precalcula la ocupación de cada alojamiento en cada mes, incluidos los meses en que no tuvo reservas (0 %). Para cada alojamiento y mes guarda las noches disponibles (habitaciones × días del mes), las noches ocupadas por reservas confirmadas o completadas y el porcentaje de ocupación. Si una estadía pasa de un mes a otro, sus noches se reparten: una estadía del 28 de enero al 3 de febrero suma 4 noches a enero y 2 a febrero. También guarda el municipio y el tipo de alojamiento, para agrupar la ocupación sin volver a las tablas base.

Política de refresco: completo (COMPLETE), bajo demanda (ON DEMAND), ejecutado por el job JOB_REFRESCO_MV_OCUPACION todos los días a las 2:00 a.m. También se puede refrescar a mano con DBMS_MVIEW.REFRESH después de una carga masiva de datos.

Justificación:

- **Completo y no rápido (FAST):** el refresco rápido exige registros de cambios (materialized view logs) en las tablas base y una consulta sencilla. Esta vista usa CONNECT BY para el calendario de meses, combinaciones externas para incluir los meses sin ocupación y COUNT(DISTINCT), y eso no lo permite. Además, una sola reserva puede cambiar varios meses, y una habitación nueva cambia el denominador de todos los meses del alojamiento.
- **Bajo demanda y no al confirmar (ON COMMIT):** con ON COMMIT, cada reserva o cambio de estado tendría que esperar a que se recalcule la ocupación de todos los alojamientos. Eso pondría el costo de un reporte de gerencia sobre la operación de recepción, que debe ser rápida.
- **Una vez al día, en la madrugada:** la ocupación mensual es un reporte para ver tendencias, y un día de atraso no cambia ninguna decisión. La disponibilidad real de una habitación no depende de la vista: la valida sp_crear_reserva contra las tablas. A las 2:00 a.m. casi nadie usa la plataforma.

5. Códigos de error propios (RAISE_APPLICATION_ERROR)

Código	Dónde	Cuándo se lanza
-20001	fn_valor_estadia, sp_crear_reserva	El check-out no es posterior al check-in
-20002	sp_crear_reserva	La habitación está ocupada en esas fechas
-20003	sp_crear_reserva	Los huéspedes superan la capacidad de la habitación
-20004	fn_valor_estadia, sp_crear_reserva	La habitación no existe
-20005	sp_crear_reserva	El cliente no existe
-20006	fn_valor_estadia	Hay noches sin temporada o sin tarifa
-20007	fn_valor_estadia	Hay noches cubiertas por dos temporadas
-20008	sp_crear_reserva	El número de huéspedes es menor que 1
-20010	trg_no_solape_reserva, trg_lineas_validar_upd, trg_no_solape_reactivar	La habitación ya está ocupada por otra reserva activa
-20011	trg_reserva_un_alojamiento, trg_lineas_validar_upd	La reserva tendría habitaciones de dos alojamientos
-20012	trg_servicio_del_alojamiento	El servicio es de otro alojamiento

-20013	trg_linea_capacidad_fechas	Los huéspedes de la línea superan la capacidad
-20014	trg_linea_capacidad_fechas	Las fechas de la línea salen del rango de la reserva
-20015	trg_resena_completada	La reseña no corresponde a una estadía completada
-20016	trg_auditoria_tarifa	No existe el administrador para registrar la auditoría
-20017	trg_temporada_sin_solape	La temporada se cruza con otra
-20020	sp_liquidacion_mensual	El mes o el año no son válidos

6. Tablas agregadas en la Entrega 2

Además de las 17 tablas del modelo, la Entrega 2 agrega:

- **LIQUIDACION_MENSUAL:** resultado del cursor de liquidación, con una fila por alojamiento, año y mes. Se relaciona con ALOJAMIENTO (un alojamiento tiene muchas liquidaciones).
- **TARIFA_PRECIO_PREVIO:** último precio conocido de cada tarifa, que usa el disparador de auditoría. Tiene una fila por cada fila de TARIFA (relación 1 a 1).
- **RESERVA_HISTORICO:** copia particionada de las reservas de los años cerrados, en el tablespace TS_HISTORICO.
- **MV_OCUPACION_MENSUAL:** vista materializada con la ocupación mensual de cada alojamiento.

7. Evidencia de pruebas

El script E2_04_Pruebas.sql prueba cada elemento de la entrega y guarda la salida en el archivo evidencia_disparadores.txt. Las pruebas que modifican datos terminan en ROLLBACK o borran lo que crearon, así que la base queda igual que antes.

Prueba	**Qué comprueba**	**Resultado esperado**
1	fn_valor_estadia: una temporada, cruce de temporadas, noches sin tarifa, recálculo de 1.000 líneas y habitación inexistente	Igual al cálculo manual; ORA-20006 y ORA-20004; 0 diferencias
2	sp_crear_reserva: reserva válida, solape, check-out igual al check-in, capacidad excedida y 0 huéspedes	Reserva creada; ORA-20002, ORA-20001, ORA-20003 y ORA-20008
3	Cursor de liquidación de julio de 2026	Totales de estadías y de servicios iguales al control
A	Auditoría de tarifa	Un UPDATE de tres tarifas deja tres filas con precio anterior y nuevo
B, C	No solape de reservas	Rechazo con ORA-20010 sobre un caso real; 0 cruces en los datos cargados

D – H	Reglas de negocio y actualizaciones	Rechazo con el código esperado en cada caso (ORA-20010 a ORA-20017)
I, J	Tablespace y particiones del histórico	Particiones en TS_HISTORICO; completitud, disyunción y reconstrucción en 0; poda de particiones; ORA-14400 para una reserva de 2026
K, L	Vista materializada y política de refresco	Diferencias en 0; la vista solo cambia al refrescarla (3 OK, 0 FALLA)